City Metro Smart Card Fare System

Assignment 1 CO1

Design Implement and Validate the Fare Logic

Individual assignment report | C++17 | Tasks 1 to 12

Student name: [NITHYA SHREE .K]

Register number: [192521145]

Programme and section: []

Course and course code: [DSA0103 OOPS WITH C++]

Faculty name: [ARUN KUMAR]

Institution: [SIMATS UNIVERSITY]

Submission date: [20-10-2026]

GitHub repository URL: [https://github.com/192521145simats-uxl/DSA0103-OOPS-with-C-/tree/main/AT's]

Repository name: DSA01-<1925211450010>-Assignment1

Scope Assumptions and Evidence

This report develops exact fare arithmetic, validates travel decisions and evaluates an extension for Airport fares, weekends and monthly passes. Integer paise and a private card balance preserve the minimum balance rule; a separate fare policy makes later rule changes manageable.

Evidence status: All outputs, test expectations and totals below are calculated or predicted unless explicitly marked as executed. Blank execution fields must be filled by the student after individual runs. No screenshot, measured timing, compiler warning or personal experiment is claimed by a placeholder. Supplied code is editable text.

Source: ASSIGNMENT 1 [CO1].pdf, 13 pages. Task numbers follow that document. The supplied SIMATS pseudocode standard itself is not attached; structured BEGIN/END, INPUT/OUTPUT, IF/ELSE and FOR notation is used and should be checked against the course handout before submission.

Explicit policy assumptions

Item	Rule used
Time	Local 24-hour time, valid hour 0 to 23 and minute 0 to 59. Peak intervals are [08:00,10:00) and [17:00,20:00).
Journey zone	The supplied zone is the farthest zone travelled. A production route service must determine it.
Concessions	Rates are not specified in the brief. Demonstration configuration uses Student 10%, Senior 20%, Adult 0%; replace with faculty-approved rates. One category applies.
Rounding	Compute peak fare first, then concession. Each percentage step rounds to nearest paise, half up, for nonnegative values.
Card	Nonnegative initial balance; accepted journey leaves at least 5000 paise. Failed debit and invalid recharge leave balance unchanged.

Item	Rule used
Task 12 weekend	Weekend discount takes priority over peak surcharge; no peak surcharge on weekend journeys. Concession then applies.
Monthly pass	Valid for one calendar month and two distinct selected fare-zone categories; both endpoints must be covered. No additional pass fare; the Rs 50 gate rule still applies. Route-wide coverage would need route-zone validation.

Report navigation

Tasks 1 to 3 Objects and invariants, constants and data types. Tasks 4 to 6 Conversions, operators and control structures. Tasks 7 to 9 Classes, safe input, implementation and tests. Tasks 10 to 12 Execution comparison, recommendations and redesign.

Task 1 Objects and Invariants

Object	Data	Behaviour and justification
SmartCard	Card ID and balance in paise	Checks and atomically applies a debit; hides the balance so unrelated code cannot bypass the rule.
Passenger	ID and category	Selects an approved concession; avoids copying passenger rules into gate code.
Journey	Origin, destination, farthest zone, local date and time	Validates journey information and requests a quote; records the decision.
Zone	Strongly typed zone category	Identifies a fare-table entry; prevents accidental use of arbitrary integers.

Object	Data	Behaviour and justification
FareTable	Base fare for each zone and rule constants	Provides a single authority for fares; validates lookup before indexing.

Encapsulation keeps SmartCard balance private. Abstraction exposes a quote or debit operation without exposing calculation details. Inheritance could give StudentPassenger and SeniorPassenger a common Passenger interface, but an enum category is simpler when only a rate differs. Polymorphism becomes useful when multiple FarePolicy implementations are selected through an interface; the initial program does not need a class hierarchy just to calculate three fares.

Correctness rules

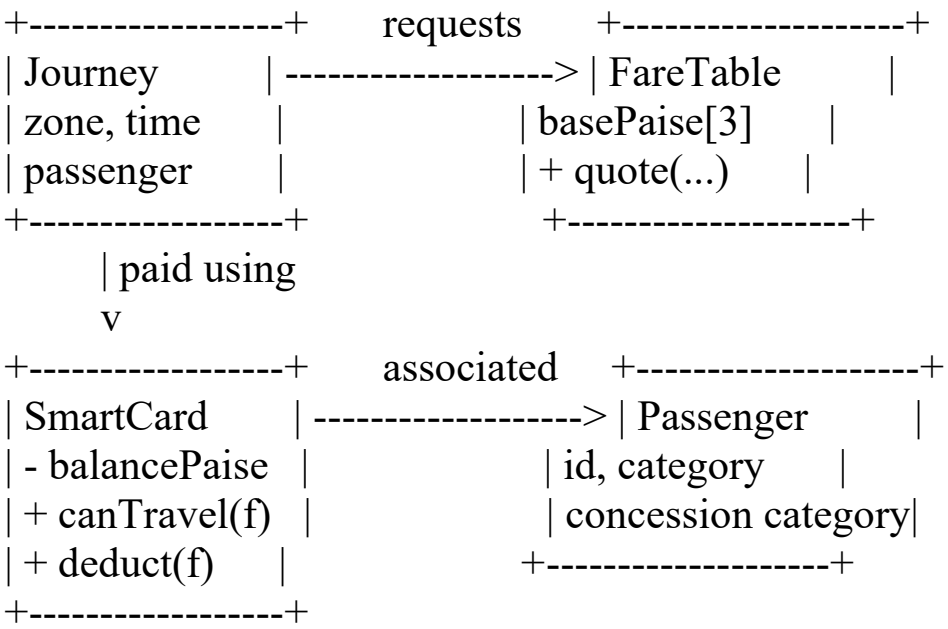
Every input zone and time must be valid before calculating a fare. Monetary values must be nonnegative and arithmetic must stay within the representation range. A successful debit changes balance by exactly the quoted fare and leaves at least Rs 50. A denied journey changes nothing. A recharge must be positive and must not overflow. The same inputs and policy version must give the same fare. In a concurrent deployment, quote validation and debit require one protected transaction to avoid two gates spending the same balance.

Procedural alternative and cost

```
long long balance = 10000; // shared state needs discipline
bool debit(long long& b, long long f) {
   if (f < 0 || b < 5000 || f > b - 5000) return false;
   b -= f; return true;
}
```

Procedural code separates records and free functions; careless code may write directly to shared state. Object-oriented code binds each balance to its controlled operations. Both take O(1) per journey and O(1) memory per card; N cards need O(N) storage. A centralized minimum-balance constant needs one edit in either style. Scattered procedural checks need O(k) edits for k duplicated checks, whereas the card method localizes that invariant. OOP improves ownership, not the asymptotic running time.

Editable UML class diagram



Legend: - private, + public; arrows are associations, not inheritance.

Task 2 Tokens Constants and Zones

```
enum class Zone { Inner, Middle, Outer };
constexpr long long BASE_PAISE[]{1000, 2000, 3000};
constexpr int PEAK_PERCENT{20};
constexpr long long MIN_BALANCE_PAISE{5000};
```

Token kind	Examples and purpose
Keywords	enum, class, constexpr, return, bool: reserved by C++.
Identifiers	Zone, BASE_PAISE, canTravel: programmer-defined names.
Constants and literals	1000, 20, true, '0': fixed literal values. A named constexpr is an identifier naming a constant.
Operators	+, >=, &&, -=, ?: perform computation or selection.
Punctuators	{ } ( ) [ ] ; , delimit declarations and statements. Some symbols also act as operators depending on context.

An identifier starts with a letter or underscore and continues with letters, digits or underscores; it cannot be a keyword. Names are case-sensitive.

Avoid implementation-reserved names, especially double underscores and leading underscore plus capital. Use descriptive class names, camelCase functions and uppercase constant names consistently.

#define BASE 1000 performs token substitution, has no C++ type or namespace scope, and may create precedence surprises unless parenthesized. const gives typed, scoped read-only objects but is not always a compile-time constant. constexpr requires a constant expression and suits a fixed fare table. enum exposes unscoped enumerators and allows integral promotion; enum class scopes enumerators and requires explicit conversion for indexing.

Macro versus constexpr lookup remains O(1). A macro itself has no object storage; a scalar constexpr often needs none when folded, but an addressable constexpr object can require storage. A three-element long long array occupies 3*sizeof(long long), typically 24 bytes, when materialized. A runtime configuration is preferable only if authorized fare changes must occur without recompilation.

Task 3 Data Types and Compatibility

Value	Recommended type	Typical size and justification
Fare and balance	long long	Usually 8 bytes; signed checked arithmetic in paise.
Hour and minute	int	Usually 4 bytes; easy validation of negative input.
Zone	enum class Zone	Usually sizeof(int); validate external code before conversion.
Peak and allowed	bool	Usually 1 byte; true or false state.
Display category	char	sizeof(char) is 1; not a money type.

Value	Recommended type	Typical size and justification
Daily journey count	long long	3,000,000 fits typical int; wider type supports aggregation.
Array index	std::size_t	Unsigned implementation-defined size; use after validation.
Approximate experiments	double	Usually 8 bytes; binary rounding unsuitable for ledger values.
float	Not used for money	Usually 4 bytes, fewer precision bits than double.
auto	Inferred from initializer	auto total=0 deduces int; auto total=0LL deduces long long.

Sizes above are typical, not measurements. C++ guarantees at least 16 value bits for int and at least 64 total bits for long long; do not assume int is always 32 bits. Obtain actual sizes and limits with the program below. Unsigned arithmetic wraps modulo its range and can hide invalid negatives; signed overflow is undefined behaviour.

```
#include <iostream>
#include <climits>
#include <limits>
int main() {
 std::cout << "bool char int long long unsigned float double\n";
 std::cout << sizeof(bool) << ' ' << sizeof(char) << ' '
 << sizeof(int) << ' ' << sizeof(long long) << ' '
 << sizeof(unsigned) << ' ' << sizeof(float) << ' '
 << sizeof(double) << '\n';
 std::cout << INT_MIN << ' ' << INT_MAX << '\n';
 std::cout << LLONG_MIN << ' ' << LLONG_MAX << '\n';
 std::cout << "double precision bits "
 << std::numeric_limits<double>::digits << '\n';
}
```

On a 32-bit int system, INT_MAX is 2,147,483,647. Even Rs 10 for each of 3,000,000 journeys totals 3,000,000,000 paise and exceeds it. Outer peak journeys total 10,800,000,000 paise. Multiplication must use 3000000LL * 3600 so the intermediate is also wide. long long safely covers these values.

In int + double, the int converts to double; in bool + int, bool promotes to int (0 or 1). Assigning the result back to an integer can discard the fractional part. Integer paise make addition exact within range. double rupees represents many decimal fractions approximately; repeated summation may drift. Both fixed-width additions are O(1), usually 8 bytes per long long or double. The error need not grow monotonically; its magnitude depends on values, count, order and implementation. Task 10 supplies a reproducible experiment instead of inventing measured drift.

Student execution evidence

Actual output: [Paste output]. Screenshot: [Insert screenshot]. Compiler and version: [Enter]. Command and date: [Enter]. Prediction versus observation: [Explain any difference].

Task 4 Implicit Conversions

Conversion inventory: a / b performs int division before assignment converts to double; casting a first converts b during division. Character arithmetic promotes char to int; assignment back to char narrows. Nonzero int converts to true. Comparing int with unsigned int of equal rank converts the signed value to unsigned. Indexing an array often converts a validated int to size_t. Percentage multiplication uses a long long operand so the int rate converts to long long.

Exact predicted output under ASCII compatible character encoding

3 3.5
C 68
1 true
s >= u

Line 1: 7/2 is 3, then 3 becomes 3.0; default formatting prints 3. Casting the numerator yields 3.5. Line 2: on ASCII systems A is 65, so ch is C and ch+1 promotes to int 68. The C++ standard does not require ASCII; character-related results depend on the execution character set. Line 3: 5 converts to true, initially printed as 1; boolalpha changes later bool

formatting to true. Line 4: -1 becomes UINT_MAX, so it is not less than unsigned 1. This comparison typically warns with -Wsign-compare under -Wextra.

```
#include <iostream>
int main() {
 int a = 7, b = 2;
 double c = a / b;
 double e = static_cast<double>(a) / b;
 std::cout << c << ' ' << e << '\n';
 char ch = 'A' + 2;
 std::cout << ch << ' ' << ch + 1 << '\n';
 bool flag = 5;
 std::cout << flag << ' ' << std::boolalpha << flag << '\n';
 unsigned int u = 1; int s = -1;
 std::cout << (s < u ? "s < u" : "s >= u") << '\n';
}
```

Explicit and safer alternative

```
int a{7}, b{2};
double c{static_cast<double>(a / b)}; // intentional truncation
double d{static_cast<double>(a) / static_cast<double>(b)};
char ch{static_cast<char>(static_cast<int>('A') + 2)};
bool flag{5 != 0};
int s{-1}; unsigned u{1};
bool less{s < 0 || static_cast<unsigned>(s) < u};
// int bad{3.7}; // ill-formed: narrowing; demonstrate separately
```

The guarded comparison expresses mathematical intent; casting s to unsigned explicitly would preserve the surprising original result. Brace initialization diagnoses many narrowing conversions, while static_cast documents intent but does not prove safety. Validate range before narrowing. Merely adding explicit casts changes neither O(1) time nor O(1) space. It catches some errors at compile time; invalid domain values still require runtime checks.

Student execution evidence

Actual output: [Paste output]. Screenshot: [Insert screenshot]. Compiler and version: [Enter]. Command and date: [Enter]. Prediction versus observation: [Explain any difference].

Task 5 Operators and Precedence

```
bool peak = (hour >= 8 && hour < 10)
        || (hour >= 17 && hour < 20);
long long withPeak = base + (peak ? base * 20 / 100 : 0);
long long finalFare = withPeak - withPeak * concession / 100;
```

Relational operators bind before &&, which binds before ||; therefore the unparenthesized peak expression is equivalent, but grouping makes the two intervals clear. && and || short-circuit left to right. Multiplication and division bind before addition or subtraction and associate left to right. The conditional operator selects one branch and associates right to left. Compound assignment binds below arithmetic and updates its left operand once; fare += surcharge is clearer than embedding changes inside larger expressions. The simple percentage example truncates fractions; the complete implementation uses an explicit half-up rule.

Exact predicted output

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

Line 1 multiplies y by 2 before adding x; parentheses force 5+3 first in the second expression. Line 2 uses old x=5 and new y=4, giving z=9; afterwards x=6 and y=4. These increments affect different objects, so there is no unsequenced conflicting access. Line 3 selects the greater value 6; % and * have equal precedence and left associativity, giving (6%4)*2=4. Line 4 integer division truncates toward zero; the remainder satisfies a=(a/b)*b+a%b and has the dividend sign. Line 5 visits i=0,3,6,9 and totals 18.

```
#include <iostream>
int main() {
 int x = 5, y = 3;
 std::cout << x+y*2 << ' ' << (x+y)*2 << '\n';
 int z=x++ + ++y;
 std::cout << z << ' ' << x << ' ' << y << '\n';
 std::cout << (x>y?x:y) << ' ' << x%4*2 << '\n';
 std::cout << 10/3 << ' ' << 10%3 << ' '
          << -10/3 << ' ' << -10%3 << '\n';
 int k=0; for(int i=0;i<10;i+=3) k+=i;
 std::cout << k << '\n';
}
```

Postfix x++ yields the old value; prefix ++x yields the updated value. Avoid i = i++: in C++17 and later the assignment is sequenced after the right operand and overwrites the increment with the old value, so i does not increase. Before C++17 this expression has undefined behaviour. Use ++i as a separate statement.

Lookup alternative

```
constexpr bool PEAK[24]{
 false,false,false,false,false,false,false,false,
 true,true,false,false,false,false,false,false,
 false,true,true,true,false,false,false,false};
// Validate 0 <= hour && hour < 24 before PEAK[hour].
```

Both forms take O(1) time. The table uses 24*sizeof(bool), commonly 24 bytes, and one indexed load; the expression uses comparisons and can short-circuit. A compiler may optimize either. The table is useful for irregular hourly schedules but does not directly model partial-hour boundaries.

Student execution evidence

Actual output: [Paste output]. Screenshot: [Insert screenshot]. Compiler and version: [Enter]. Command and date: [Enter]. Prediction versus observation: [Explain any difference].

Task 6 Control Structures and TODO Completions

Use a switch for discrete zones, if/else for validation and gate decisions, and ?: for a short choice such as a surcharge amount. Without a matching case and without default, a switch executes no case body; an uninitialized

fare would then be unsafe. A default rejection makes invalid enum values explicit. break exits a switch case or loop; continue skips an invalid journey and starts the next iteration. A range-based for suits an existing batch, a counted for suits generated journeys and a while suits a live stream whose length is unknown.

Structured fare pseudocode

```
ALGORITHM ProcessJourneys
INPUT journey list, initial balance
OUTPUT fare and decision for each journey, final balance
BEGIN
  FOR EACH journey IN journey list DO
    IF zone or time is invalid THEN
      OUTPUT invalid journey
      CONTINUE
    END IF
    SELECT zone
      CASE Inner: base <- 1000
      CASE Middle: base <- 2000
      CASE Outer: base <- 3000
    END SELECT
    peak <- (08:00 <= time < 10:00) OR (17:00 <= time < 20:00)
    fare <- base
    IF peak THEN fare <- fare + base * 20 DIV 100 END IF
    IF balance >= 5000 AND fare <= balance - 5000 THEN
      balance <- balance - fare
      OUTPUT allowed, fare, balance
    ELSE
      OUTPUT denied, balance
    END IF
  END FOR
END
```

Completed supplied program

```
#include <iomanip>
#include <iostream>
enum class Zone { Inner, Middle, Outer };
constexpr int BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;
```

```
long long fare(Zone z, bool peak) {
 // TODO 1 completed
 long long base = 0;
 switch (z) {
 case Zone::Inner: base = BASE_PAISE[0]; break;
 case Zone::Middle: base = BASE_PAISE[1]; break;
 case Zone::Outer: base = BASE_PAISE[2]; break;
 default: return -1; // invalid fare sentinel rejected below
 }
 return base + (peak ? base*PEAK_SURCHARGE_PERCENT/100 : 0);
}
bool canTravel(long long balance, long long f) {
 // TODO 2 completed; subtraction cannot overflow after these guards
 return f >= 0 && balance >= MIN_BALANCE_PAISE
     && f <= balance - MIN_BALANCE_PAISE;
}
int main() {
 long long balance = 10000;
 struct Trip { Zone zone; bool peak; };
 Trip trips[] = {{Zone::Inner, false}, {Zone::Outer, true},
                 {Zone::Middle, false}};
 for (const Trip& t : trips) {
  long long f = fare(t.zone, t.peak);
  if (canTravel(balance, f)) {
   balance -= f;
   std::cout << "Fare Rs " << f / 100 << '.' << std::setw(2)
    << std::setfill('0') << f % 100 << std::setfill(' ');
  } else {
   std::cout << "Denied (low balance)";
  }
  std::cout << "  balance Rs " << balance / 100 << '\n';
 }
}
```

Predicted output

```
Fare Rs 10.00  balance Rs 90
Fare Rs 36.00  balance Rs 54
Denied (low balance)  balance Rs 54
```

Trace: 10000-1000=9000; 9000-3600=5400; a further 2000 would leave 3400, so it is denied. The supplied Trip stores only a peak bool, so exact-hour validation belongs in an additional driver using isPeak(10,0) and

isPeak(20,0); both must be false. Use a fresh Rs 100 card for each boundary test to avoid confusing a time result with low-balance denial.

```
// Additional boundary check in a separate copy of the driver.
// Keep the original supplied main unchanged for its required run.
bool isPeakHour(int hour) {
 return (hour>=8 && hour<10) || (hour>=17 && hour<20);
}
// Inside the additional driver's main, with the same fare function:
// for(int hour : {10,20}) {
//   long long balance=10000;
//   long long f=fare(Zone::Inner,isPeakHour(hour));
//   if(canTravel(balance,f)) balance-=f;
//   std::cout << hour << ' ' << f << ' ' << balance << '\n';
// }
// Predicted: 10 1000 9000, then 20 1000 9000 (paise).
```

Array alternative: validate the zone index and return FARE[index]. For three or four zones, both switch and lookup are O(1); a switch may compile into a jump table. A table needs z*sizeof(long long), typically 24 bytes for three entries. With z variable, table storage is O(z). A fourth zone adds an enum and table entry, whereas a switch also needs another case. Task 9 provides the hour-boundary driver and Task 12 expands the table.

Student execution evidence

Actual output: [Paste output]. Screenshot: [Insert screenshot]. Compiler and version: [Enter]. Command and date: [Enter]. Prediction versus observation: [Explain any difference].

Task 7 Procedural and Object Oriented Versions

Working procedural version

```
#include <iostream>
constexpr long long MIN_BALANCE{5000};
bool canTravel(long long b, long long f) {
 return f>=0 && b>=MIN_BALANCE && f<=b-MIN_BALANCE;
}
bool deduct(long long& b, long long f) {
 if (!canTravel(b,f)) return false;
 b-=f; return true;
}
int main() {
 long long balance{10000};
 for (long long f : {1000LL,3600LL,2000LL})
  std::cout << deduct(balance,f) << ' ' << balance << '\n';
}
```

Working class version

```
#include <iostream>
#include <stdexcept>
class SmartCard {
 long long balance;
 static constexpr long long minimum{5000};
public:
 explicit SmartCard(long long b):balance{b} {
  if(b<0) throw std::invalid_argument("negative balance");
 }
 bool canTravel(long long f) const {
  return f>=0 && balance>=minimum && f<=balance-minimum;
 }
 bool deduct(long long f) {
  if(!canTravel(f)) return false;
  balance-=f; return true;
 }
 long long getBalance() const {return balance;}
};
int main() {
 SmartCard card{10000};
 for(long long f : {1000LL,3600LL,2000LL})
  std::cout << card.deduct(f) << ' '
            << card.getBalance() << '\n';
}
```

Predicted output for both versions

```
1 9000
1 5400
0 5400
```

The class repeats its check inside deduct, so callers cannot accidentally skip it. A private balance prevents card.balance = -1 from compiling; a public member permits this assignment and breaks the invariant immediately. A centralized constant makes a minimum-balance revision a one-place change in both examples. The class makes enforcement easier to audit. Neither privacy nor ordinary nonvirtual methods add per-card payload or change O(1) time/space. This class does not by itself provide thread safety or persistent transaction durability.

Student execution evidence

Actual output: [Paste output]. Screenshot: [Insert screenshot]. Compiler and version: [Enter]. Command and date: [Enter]. Prediction versus observation: [Explain any difference].

Task 8 Safe Input and Money Output

```
#include <iostream>
#include <sstream>
#include <string>
#include <limits>
#include <iomanip>
bool readHour(int& hour) {
 for (;;) {
  std::cout << "Hour 0..23: ";
  if (!(std::cin >> hour)) {
   if (std::cin.eof()) return false;
   std::cin.clear();
   std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
   std::cout << "Invalid number\n"; continue;
  }
  std::string tail; std::getline(std::cin, tail);
  if (tail.find_first_not_of(" \t\r") != std::string::npos
      || hour < 0 || hour > 23) {
   std::cout << "Invalid hour\n"; continue;
  }
  return true;
 }
}
void money(long long paise) { // precondition: paise >= 0
 std::cout << "Rs " << paise/100 << '.' << std::setw(2)
  << std::setfill('0') << paise%100 << std::setfill(' ');
}
int main() {
 int hour{}; if (!readHour(hour)) return 0;
 std::cout << "Accepted " << hour << "  Balance ";
 money(5400); std::cout << '\n';
}
```

For input lines abc, 25, 10, the predicted messages are Invalid number, Invalid hour, then Accepted 10 Balance Rs 54.00, with a prompt before

each attempt. fail() signals extraction failure, clear() resets the stream state, and ignore() removes the offending line. EOF exits instead of looping forever. Checking the rest of the line also rejects 10abc, which a bare integer extraction would partly accept.

Line oriented alternative

```
std::string line;
if (std::getline(std::cin, line)) {
 try {
  std::size_t used{};
  int hour{std::stoi(line, &used)};
  if (line.find_first_not_of(" \t\r", used) != std::string::npos
      || hour < 0 || hour > 23)
   throw std::invalid_argument("hour");
  // valid hour
 } catch (const std::exception&) {
  std::cout << "Invalid hour\n";
 }
}
```

Include <string>, <stdexcept> and <iostream> for the alternative. stoi can throw invalid_argument or out_of_range; its used index must be checked to reject trailing junk. Each approach scans O(L) characters for a line of length L. getline requires O(L) buffer memory. Direct extraction with discard can use O(1) auxiliary space, but this strict version retains the trailing suffix and therefore uses O(L) too. Impose an input-length limit for hostile streams.

For floating displays, fixed << setprecision(2) controls presentation only; it does not repair the stored value. setw affects the next field, while setfill persists until reset. Integer division and a padded two-digit remainder print ledger values exactly. A journey report should show journey ID, zone, time, quoted fare, accepted/denied status and remaining balance in consistent columns.

Student execution evidence

Actual output: [Paste output]. Screenshot: [Insert screenshot]. Compiler and version: [Enter]. Command and date: [Enter]. Prediction versus observation: [Explain any difference].

Task 9 Complete Fare Implementation and Tests

The following standalone program implements all original fare rules, configurable concessions, strict zone/time validation, safe recharge and table-driven checks. Money remains in paise. The displayed test outcomes are expected, not claims that the student has executed them. Task 8 handles textual input before passing validated numbers to this domain layer.

```
#include <array>
#include <iostream>
#include <iomanip>
#include <stdexcept>
#include <limits>
using Money = long long;
enum class Zone { Inner, Middle, Outer };
enum class Category { Adult, Student, Senior };
constexpr std::array<Money,3> BASE{{1000,2000,3000}};
constexpr Money MINIMUM{5000};
constexpr int PEAK_PERCENT{20};
constexpr int STUDENT_PERCENT{10}; // assumed demonstration rate
constexpr int SENIOR_PERCENT{20};  // assumed demonstration rate
bool valid(Zone z) {
 int i=static_cast<int>(z); return i>=0 && i<3;
}
bool isPeak(int hour, int minute) {
 if(hour<0 || hour>23 || minute<0 || minute>59)
  throw std::invalid_argument("time");
 int t=hour*60+minute;
 return (t>=480 && t<600) || (t>=1020 && t<1200);
}
// Round a nonnegative percentage half up without amount*percent
overflow.
Money percent(Money amount, int rate) {
 if(amount<0 || rate<0 || rate>100)
  throw std::invalid_argument("percentage");
 return (amount/100)*rate + ((amount%100)*rate+50)/100;
}
int concession(Category c) {
 switch(c) {
 case Category::Adult: return 0;
```

```
 case Category::Student: return STUDENT_PERCENT;
 case Category::Senior: return SENIOR_PERCENT;
 }
 throw std::invalid_argument("category");
}
Money quote(Zone z, int hour, int minute, Category c) {
 if(!valid(z)) throw std::invalid_argument("zone");
 Money f=BASE[static_cast<std::size_t>(z)];
 if(isPeak(hour,minute)) f+=percent(f,PEAK_PERCENT);
 return f-percent(f,concession(c));
}
class SmartCard {
 Money balance;
public:
 explicit SmartCard(Money b):balance{b} {
  if(b<0) throw std::invalid_argument("balance");
 }
 Money getBalance() const {return balance;}
 bool canTravel(Money f) const {
  return f>=0 && balance>=MINIMUM && f<=balance-MINIMUM;
 }
 bool deduct(Money f) {
  if(!canTravel(f)) return false;
  balance-=f; return true;
 }
 bool recharge(Money amount) {
  if(amount<=0 || amount>std::numeric_limits<Money>::max()-balance)
   return false;
  balance+=amount; return true;
 }
};
void printMoney(Money f) {
 std::cout << "Rs " << f/100 << '.' << std::setw(2)
  << std::setfill('0') << f%100 << std::setfill(' ');
}
int main() {
 struct Test {Zone z; int h,m; Category c; Money want;};
 const Test tests[]{
  {Zone::Inner,7,59,Category::Adult,1000},
  {Zone::Middle,7,59,Category::Adult,2000},
```

```
 {Zone::Outer,7,59,Category::Adult,3000},
 {Zone::Inner,8,0,Category::Adult,1200},
 {Zone::Middle,9,59,Category::Adult,2400},
 {Zone::Outer,17,0,Category::Adult,3600},
 {Zone::Inner,10,0,Category::Adult,1000},
 {Zone::Outer,20,0,Category::Adult,3000},
 {Zone::Outer,19,59,Category::Adult,3600},
 {Zone::Middle,8,0,Category::Student,2160},
 {Zone::Outer,17,0,Category::Senior,2880}};
int failures=0, number=0;
auto check=[&](bool ok) {
 ++number; if(!ok) ++failures;
 std::cout << "T" << number << ' ' << (ok?"PASS":"FAIL") << '\n';
};
for(const auto& t: tests) check(quote(t.z,t.h,t.m,t.c)==t.want);
SmartCard exact{6200}; check(exact.deduct(1200)
                    && exact.getBalance()==5000);
SmartCard shortCard{6199}; check(!shortCard.deduct(1200)
                        && shortCard.getBalance()==6199);
check(!shortCard.recharge(-100) && shortCard.getBalance()==6199);
check(!shortCard.deduct(-1));
try {quote(static_cast<Zone>(99),8,0,Category::Adult); check(false);}
catch(const std::invalid_argument&) {check(true);}
try {quote(Zone::Inner,24,0,Category::Adult); check(false);}
catch(const std::invalid_argument&) {check(true);}
try {quote(Zone::Inner,8,60,Category::Adult); check(false);}
catch(const std::invalid_argument&) {check(true);}
SmartCard full{std::numeric_limits<Money>::max()};
check(!full.recharge(1));
SmartCard card{20000}; Money total=0;
const Test five[]{
 {Zone::Inner,7,0,Category::Adult,1000},
 {Zone::Middle,8,0,Category::Adult,2400},
 {Zone::Outer,10,0,Category::Adult,3000},
 {Zone::Inner,17,0,Category::Adult,1200},
 {Zone::Middle,20,0,Category::Adult,2000}};
for(const auto& t: five) {
 Money f=quote(t.z,t.h,t.m,t.c);
 if(card.deduct(f)) total+=f; else ++failures;
}
```

```
 check(total==9600 && card.getBalance()==10400);
 std::cout << "Five journey total "; printMoney(total);
 std::cout << "  balance "; printMoney(card.getBalance());
 std::cout << "\nFailures " << failures << '\n';
 return failures==0?0:1;
}
```

Expected test matrix

Case	Input	Calculated expectation	Student actual
T1 to T3	Each zone at 07:59, adult	1000, 2000, 3000 paise	[Enter]
T4 to T6	Inner 08:00, Middle 09:59, Outer 17:00	1200, 2400, 3600 paise	[Enter]
T7 to T9	Inner 10:00, Outer 20:00, Outer 19:59	1000, 3000, 3600 paise	[Enter]
T10 T11	Student Middle peak; Senior Outer peak	2160; 2880 paise using assumed rates	[Enter]
T12	6200 balance, 1200 fare	Allowed; 5000 remains	[Enter]
T13	6199 balance, 1200 fare	Denied; 6199 unchanged	[Enter]
T14 T15	Negative recharge; negative fare	Rejected; unchanged balance	[Enter]
T16 to T18	Zone 99, hour 24, minute 60	invalid_argument, no debit	[Enter]
T19	LLONG_MAX balance, recharge 1	Rejected without overflowing	[Enter]

Case	Input	Calculated expectation	Student actual
T20	Five journeys from 20000 paise	Total 9600, balance 10400	[Enter]
Input test	Task 8: abc then 10	Recover and accept 10	[Enter]

Five journey hand calculation

Journey	Fare Rs	Balance Rs
Initial	0.00	200.00
Inner 07:00	10.00	190.00
Middle 08:00	24.00	166.00
Outer 10:00	30.00	136.00
Inner 17:00	12.00	124.00
Middle 20:00	20.00	104.00
Total charged	96.00	104.00

Predicted driver output: T1 PASS through T20 PASS, then Five journey total Rs 96.00 balance Rs 104.00 and Failures 0. For t fixed-width cases, a table-driven test loop takes O(t) time and O(t) stored cases; the loop itself uses O(1) auxiliary space. Separate checks also take O(t) time and O(t) source size but need no case array. A new boundary is one row in the table. This finite matrix supports the invariants but is not a proof for every input; the guards explain correctness for other representable values.

Student execution evidence
Actual output: [Paste output]. Screenshot: [Insert screenshot]. Compiler and version: [Enter]. Command and date: [Enter]. Prediction versus observation: [Explain any difference].

Task 10 Expected Totals and Execution Comparison
Two workloads separate range from precision. Workload A uses the actual adult Outer peak fare of Rs 36.00. With a usual IEEE 754 binary64 double, 36 and all listed integer totals are exactly representable, so both methods should agree exactly. Workload B deliberately accumulates Rs 0.10 to expose decimal representation error; it is a precision stress test, not a metro fare. Current configured metro fares, including the assumed concessions, are multiples of Rs 0.20 and some are not exactly representable in binary.

Journeys	A exact paise	A exact rupees	32 bit int paise safe
20,000	72,000,000	720,000.00	Yes
500,000	1,800,000,000	18,000,000.00	Yes
3,000,000	10,800,000,000	108,000,000.00	No

Journeys	B exact paise	B exact rupees	Measured double error Rs
20,000	200,000	2,000.00	[Run and enter]
500,000	5,000,000	50,000.00	[Run and enter]

Journeys	B exact paise	B exact rupees	Measured double error Rs
3,000,000	30,000,000	300,000.00	[Run and enter]

```
#include <iostream>
#include <iomanip>
#include <climits>
#include <chrono>
int main() {
 std::cout << std::setprecision(17);
 for(long long n : {20000LL,500000LL,3000000LL}) {
  long long paise=0; double rupees=0.0;
  auto start=std::chrono::steady_clock::now();
  for(long long i=0;i<n;++i) {paise+=3600;rupees+=36.0;}
  auto stop=std::chrono::steady_clock::now();
  double seconds=std::chrono::duration<double>(stop-start).count();
  std::cout << "A " << n << " paise=" << paise
   << " double=" << rupees
   << " error=" << rupees-static_cast<double>(paise)/100.0
   << " int_fits=" << (paise<=INT_MAX)
   << " seconds=" << seconds << '\n';
  paise=0;rupees=0;
  for(long long i=0;i<n;++i) {paise+=10;rupees+=0.10;}
  std::cout << "B " << n << " paise=" << paise
   << " double=" << rupees
   << " error=" << rupees-static_cast<double>(paise)/100.0
   << '\n';
 }
}
```

The INT_MAX comparison predicts overflow without executing undefined signed overflow. For a 32-bit int and 3600-paise fare, at most floor(2,147,483,647/3600)=596,523 journeys fit; the 596,524th addition cannot be represented. There is no portable exact wrapped result to report. A long long has ample range for the network totals. A binary64 double

represents all integers through 2^53 exactly, but decimal fractions can already be inexact at very small values.

Accumulator	Time per addition	Typical bytes	Failure mechanism
int paise	O(1)	4	Undefined signed overflow beyond INT_MAX
long long paise	O(1)	8	Exact until range exhausted; guard first
double rupees	O(1)	8	Binary rounding and loss of low digits as totals grow

Measurement and warning record

Run with g++ -std=c++17 -Wall -Wextra -Wconversion -O2 totals10.cpp -o totals10. Record compiler version, CPU, build flags and five repetitions; report median seconds and min/max. Timing printed here measures the combined A loop, not separate integer and double speed. For a fair speed comparison, use separate loops, identical counts, consumed outputs and the same optimization settings. Do not infer a deployment throughput from one laptop measurement.

Program	Expected diagnostic or observation	Actual compiler message
Task 4	Signed/unsigned comparison warning likely under -Wextra	[Paste exact diagnostic]
int x{3.7}	Compile-time narrowing error; keep in a separate negative test	[Paste]
Task 9 and 12	No warning intended with stated flags	[Paste or record none]
Network accumulator	No runtime overflow experiment needed; range calculation rejects int	[Record range check]

The exact numerical drift from workload B must be taken from the run, not guessed. Formatting only two decimal places can conceal it. Other differences can come from character encoding, platform type widths, compiler mode, rounding choices or invalid inputs. Do not describe an unsigned modulo result as the defined result of signed overflow.

Student execution evidence

Actual output: [Paste output]. Screenshot: [Insert screenshot]. Compiler and version: [Enter]. Command and date: [Enter]. Prediction versus observation: [Explain any difference].

Task 11 Recommended Final Design

Recommend long long paise, enum class zones, a centralized constexpr fare table, explicit local-time validation, and a SmartCard with private balance and guarded recharge/debit. The calculated boundary cases show that the chosen half-open intervals exclude 10:00 and 20:00, and the debit test permits exactly Rs 50 remaining while rejecting one paisa less. The network range calculation rules out a 32-bit int ledger. These are analytical grounds; the final experimental defence must also cite the student execution evidence below.

A fare revision changes one authoritative table and any affected policy constants, then reruns the six zone/time combinations, concession tests, minimum-balance boundaries and aggregate checks. Version the policy and deploy it consistently. Keep a transaction record with the charged amount and policy version so older transactions remain explainable after a revision.

Limits: the original system does not model route distances, transfers, refunds, lost cards, offline reconciliation, authenticated fare updates, persistence or concurrent gates. Category eligibility needs verification. A production system needs transactional debit and idempotent journey IDs, not merely private fields. Computation remains O(1) per journey and a streamed day is O(n) time with O(1) working memory; storing every journey takes O(n). Correct integer arithmetic scales to the stated three million journeys, while actual capacity must be measured.

Configuration file alternative

```
BEGIN LoadFarePolicy
  READ policy version and zone count
  FOR EACH configured zone DO
    READ zone identifier and nonnegative paise fare
    REJECT duplicate, missing, malformed or out of range value
  END FOR
  VALIDATE surcharge, concessions and complete zone coverage
  ATOMICALLY publish the complete validated policy
  ON failure retain last approved policy or disable new journeys
END
```

Loading z fare values takes O(z) time and O(z) storage, or O(L) parsing time for L file characters. Once loaded, lookup stays O(1). Compiled constants avoid file parsing and runtime trust concerns but every revision

needs rebuild, validation and redeployment. Configuration allows faster rule rollout, but requires authentication, versioning and consistent activation; it should never apply a partially read fare table.

Student experimental defence to complete
[Cite the Task 9 observed test count and any failures, Task 10 measured exact totals and double drift, compiler warnings resolved, and Task 12 observed results. Explain whether these observations support the recommendation. Do not present the calculated predictions as your own measured experiments.]

Task 12 Reevaluation and Adapted Design

A Impact analysis

Element	Required change
Zone and fare constants	Add Airport and 6000 paise; expand and validate all indexing.
Peak and weekend expressions	Use weekend branch before peak so modifiers cannot stack.
Concessions	Keep rates configurable; apply after the selected temporal adjustment.
Switches	Add Airport wherever enumerators were handled or replace fare switch with validated table lookup.
Journey	Add origin, destination, farthest zone and validated calendar date.
Pass	Store two distinct zone categories and year/month validity; check both endpoints and declared coverage semantics.
SmartCard	Retain guarded balance, add controlled optional pass ownership; never expose a public balance.

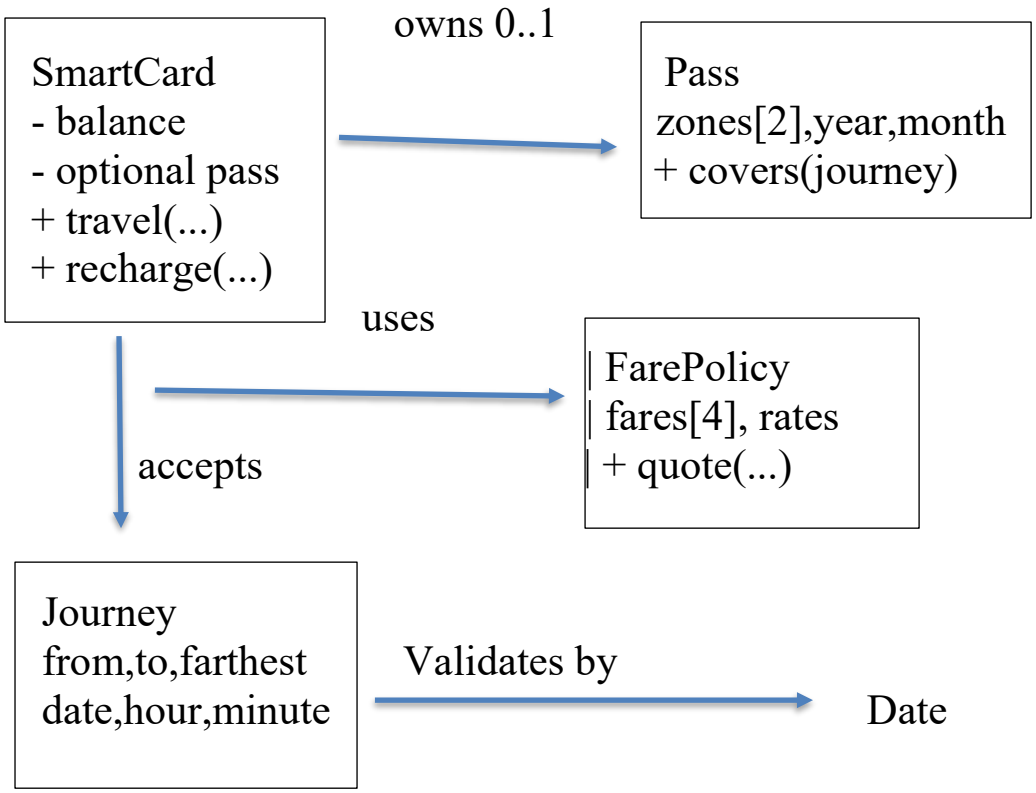
Element	Required change
Tests and reports	Add Airport, weekend precedence, expiry, uncovered route and zero-fare minimum-balance cases.

B Evaluation of the original design

The original switch can accept Airport by adding a case, so it is not technically unusable. However, mixing zone selection, weekend priorities, concessions and pass checks into one growing switch duplicates policy decisions. Retain the card invariant, modify the zone representation and redesign fare calculation as a separate policy with a pass value object. Full inheritance is unnecessary for four fare values.

Worked calculations for an adult: Airport weekday off-peak Rs 60; weekday peak Rs 72; weekend peak Rs 54 because the 10% weekend discount replaces peak surcharge. A student weekend Airport fare is Rs 48.60 under the assumed 10% concession. With Rs 120 on a card, a Rs 72 fare is denied because Rs 48 would remain, whereas Rs 70 is the exact starting balance needed for a Rs 20 fare. A covered, valid monthly pass yields zero fare, but a card below Rs 50 is still denied under the stated interpretation.

Editable revised UML diagram



*-- composition; + public; - private; arrays shown as attributes.

Revised policy pseudocode

```
ALGORITHM RevisedQuote
INPUT validated journey, category, optional pass
OUTPUT exact nonnegative fare in paise
BEGIN
  VALIDATE zones, calendar date, hour and minute
  IF pass is valid for journey month AND covers both endpoints THEN
    RETURN 0
  END IF
  fare <- fare table[farthest zone]
  IF journey date is weekend THEN
    fare <- fare - rounded 10 percent of fare
  ELSE IF journey time is peak THEN
    fare <- fare + rounded 20 percent of fare
  END IF
  fare <- fare - rounded category concession of fare
  RETURN fare
END
ALGORITHM Travel
BEGIN
  fare <- RevisedQuote(journey, category, card pass)
  IF fare <= balance - minimum AND balance >= minimum THEN
    balance <- balance - fare
    RETURN allowed
  END IF
  RETURN denied
END
```

C Standalone revised implementation

```
#include <array>
#include <optional>
#include <iostream>
#include <stdexcept>
#include <limits>
using Money=long long;
enum class Zone {Inner,Middle,Outer,Airport};
enum class Category {Adult,Student,Senior};
constexpr std::array<Money,4> FARES{{1000,2000,3000,6000}};
constexpr Money MINIMUM{5000};
constexpr int PEAK_RATE{20}, WEEKEND_RATE{10};
```

```
constexpr int STUDENT_RATE{10}, SENIOR_RATE{20}; // assumed
bool valid(Zone z) {int i=static_cast<int>(z);return i>=0 && i<4;}
struct Date {
 int year,month,day;
 bool valid() const {
  if(year<1900 || year>9999 || month<1 || month>12) return false;
  const int days[]{31,28,31,30,31,30,31,31,30,31,30,31};
  bool leap=year%400==0 || (year%4==0 && year%100!=0);
  int maximum=days[month-1]+(month==2 && leap?1:0);
  return day>=1 && day<=maximum;
 }
 bool weekend() const { // Gregorian calendar: Sunday=0
  if(!valid()) throw std::invalid_argument("date");
  const int offsets[]{0,3,2,5,0,3,5,1,4,6,2,4};
  int y=year-(month<3?1:0);
  int w=(y+y/4-y/100+y/400+offsets[month-1]+day)%7;
  return w==0 || w==6;
 }
};
struct Journey {Zone from,to,farthest;Date date;int hour,minute;};
void validate(const Journey& j) {
 if(!valid(j.from)||!valid(j.to)||!valid(j.farthest)
  || !j.date.valid() || j.hour<0 || j.hour>23
  || j.minute<0 || j.minute>59)
  throw std::invalid_argument("journey");
 // For this ordered-zone model, farthest cannot precede an endpoint.
 if(static_cast<int>(j.farthest)<static_cast<int>(j.from)
  || static_cast<int>(j.farthest)<static_cast<int>(j.to))
  throw std::invalid_argument("farthest zone");
}
class Pass {
 Zone first,second;int year,month;
public:
 Pass(Zone a,Zone b,int y,int m):first{a},second{b},year{y},month{m} {
  if(!valid(a)||!valid(b)||a==b||y<1900||y>9999||m<1||m>12)
   throw std::invalid_argument("pass");
 }
 bool covers(const Journey& j) const {
  auto selected=[&](Zone z){return z==first || z==second;};
  return j.date.year==year && j.date.month==month
```

```
      && selected(j.from) && selected(j.to);
 }
};
Money percent(Money a,int r) {
 if(a<0||r<0||r>100) throw std::invalid_argument("percentage");
 return a/100*r+((a%100)*r+50)/100;
}
int concession(Category c) {
 switch(c) {
 case Category::Adult:return 0;
 case Category::Student:return STUDENT_RATE;
 case Category::Senior:return SENIOR_RATE;
 }
 throw std::invalid_argument("category");
}
class FarePolicy {
public:
 Money quote(const Journey& j,Category c,const std::optional<Pass>& p)
const {
  validate(j); int rate=concession(c);
  if(p && p->covers(j)) return 0;
  Money f=FARES[static_cast<std::size_t>(j.farthest)];
  int t=j.hour*60+j.minute;
  if(j.date.weekend()) f-=percent(f,WEEKEND_RATE);
  else if((t>=480&&t<600)||(t>=1020&&t<1200))
   f+=percent(f,PEAK_RATE);
  return f-percent(f,rate);
 }
};
class SmartCard {
 Money balance;std::optional<Pass> pass;
public:
 explicit SmartCard(Money b,std::optional<Pass> p=std::nullopt)
  :balance{b},pass{p} {
  if(b<0) throw std::invalid_argument("balance");
 }
 bool canTravel(Money f) const {
  return f>=0 && balance>=MINIMUM && f<=balance-MINIMUM;
 }
 bool travel(const Journey& j,Category c,const FarePolicy& policy) {
```

```
 Money f=policy.quote(j,c,pass);
 if(!canTravel(f)) return false;
 balance-=f;return true;
}
bool recharge(Money amount) {
 if(amount<=0||amount>std::numeric_limits<Money>::max()-balance)
  return false;
 balance+=amount;return true;
}
Money getBalance() const {return balance;}
};
int main() {
FarePolicy policy; int count=0,failures=0;
auto check=[&](bool ok) {
 ++count; if(!ok)++failures;
 std::cout << "R" << count << ' ' << (ok?"PASS":"FAIL") << '\n';
};
Journey
weekday{Zone::Inner,Zone::Airport,Zone::Airport,{2026,10,19},8,0};
Journey weekend=weekday;weekend.date={2026,10,17};
check(policy.quote(weekday,Category::Adult,std::nullopt)==7200);
check(policy.quote(weekend,Category::Adult,std::nullopt)==5400);
check(policy.quote(weekend,Category::Student,std::nullopt)==4860);
weekday.hour=10;
check(policy.quote(weekday,Category::Adult,std::nullopt)==6000);
weekday.hour=20;
check(policy.quote(weekday,Category::Adult,std::nullopt)==6000);
Pass october{Zone::Inner,Zone::Airport,2026,10};
check(policy.quote(weekday,Category::Adult,october)==0);
Journey november=weekday;november.date={2026,11,2};
check(policy.quote(november,Category::Adult,october)==6000);
Journey uncovered=weekday;uncovered.to=Zone::Outer;
check(policy.quote(uncovered,Category::Adult,october)==6000);
SmartCard low{4999,october};

check(!low.travel(weekday,Category::Adult,policy)&&low.getBalance()==
=4999);
SmartCard exact{5000,october};
```

```
check(exact.travel(weekday,Category::Adult,policy)&&exact.getBalance(
)==5000);
 SmartCard paid{10400};

check(paid.travel(weekend,Category::Adult,policy)&&paid.getBalance()=
=5000);
 try {Pass bad{Zone::Inner,Zone::Inner,2026,10};(void)bad;check(false);}
 catch(const std::invalid_argument&) {check(true);}
 try {Journey bad=weekday;bad.date={2026,2,30};
     policy.quote(bad,Category::Adult,std::nullopt);check(false);}
 catch(const std::invalid_argument&) {check(true);}
 check(!paid.recharge(-1)&&paid.getBalance()==5000);
 std::cout << "Failures " << failures << '\n';
 return failures==0?0:1;
}
```

D Test expectations and defence

Test	Changed requirement	Calculated expected result	Student actual
R1	Airport weekday 08:00	7200 paise	[Enter]
R2	Airport Saturday 08:00	5400 paise; no peak surcharge	[Enter]
R3	Student weekend Airport	4860 paise	[Enter]
R4 R5	Airport weekday 10:00 and 20:00	6000 paise each	[Enter]
R6	Valid October Inner Airport pass	Zero fare	[Enter]
R7	October pass used November 2	6000 paise	[Enter]

Test	Changed requirement	Calculated expected result	Student actual
R8	Destination Outer not selected	6000 paise using farthest Airport input	[Enter]
R9 R10	Pass balance 4999 and 5000	Deny unchanged; allow unchanged	[Enter]
R11	10400 balance, weekend Airport	Allow; 5000 remains	[Enter]
R12	Same zone selected twice	Reject pass	[Enter]
R13	February 30	Reject date	[Enter]
R14	Negative recharge	Reject unchanged	[Enter]

Predicted output: R1 PASS through R14 PASS, then Failures 0. October 19 2026 is Monday and October 17 is Saturday under the Gregorian calendar implemented above. The calculation and pass checks are O(1) for four fixed zones and two pass zones. Fare-table space is 4*sizeof(long long), typically 32 bytes, versus typically 24 bytes originally; optional pass ownership adds a fixed-size object per card. With z configurable zones, policy storage is O(z); if a pass stores k zones, a linear membership check takes O(k), while a fixed-domain bitset can provide constant-time membership at O(z) bits.

Decision: retain the card invariant, modify zone/table data and separate the temporal and pass policies. The worked examples show why weekend and peak must be mutually exclusive and why pass validity needs its own object. The extra date/pass state and explicit precedence are justified by the changed requirements. A switch could still work, but separating policy prevents each zone case from repeating discount logic. The defence remains provisional until the student records observed results; failures should trigger a rule or code correction followed by a fresh run.

Coverage limitation: the brief does not define whether a pass covers endpoints, farthest zone or every traversed zone. This implementation uses the two selected endpoint categories. For route-wide coverage, extend Journey with all traversed zones and require every one to be covered; checking r traversed zones then costs O(r). Pass purchase price, refunds, calendar timezone and eligibility verification are outside the stated requirements.

Student execution evidence

Actual output: [Paste output]. Screenshot: [Insert screenshot]. Compiler and version: [Enter]. Command and date: [Enter]. Prediction versus observation: [Explain any difference].

Execution and Submission Record

Create DSA01-<RegisterNumber>-Assignment1 with Task01 to Task12 folders. Each folder must contain the relevant source, inputs, observed output and screenshots, including explanatory tasks. Save the original predictions before execution and commit each task progressively. Do not claim that this report substitutes for the mandatory individual execution or progressive GitHub history.

```
DSA01-<RegisterNumber>-Assignment1/
  README.md
  Task01/ ... Task12/
    source.cpp
    input.txt
    prediction.txt
    actual-output.txt
    screenshots/
  report.docx
```

Suggested source mapping: Task03 types.cpp; Task04 predict4.cpp; Task05 predict5.cpp; Task06 todo6.cpp plus a boundary driver; Task07 procedural7.cpp and oop7.cpp; Task08 input8.cpp; Task09 fare9.cpp; Task10 totals10.cpp; Task12 redesign12.cpp. For Tasks 1, 2 and 11 execute small demonstrations of the discussed invariants, constants and revised configuration rather than leaving those folders empty.

Build and run example

```
g++ --version
g++ -std=c++17 -Wall -Wextra -Wconversion -O2 fare9.cpp -o fare9
./fare9
# Windows PowerShell uses .\fare9.exe
# Repeat separately for each standalone source file.
# Task 12 requires a compiler/library with C++17 std::optional support.
```

Required README field	Student entry
Name and register number	[Enter]
Compiler and version	[Paste g++ --version]

Required README field	Student entry
Compile and run command per source	[Enter exact commands]
Result per task	[Enter observed result and limitations]
GitHub URL and faculty access	[Enter and verify]
Progressive commits	[At least one commit per task; record hashes]
Measurements	[Machine, flags, dataset, repetitions, totals and timings]

Final review

Replace student details and approved concession rates. Execute each source individually. Preserve predictions before running. Enter actual results and explain discrepancies. Insert legible output screenshots. Add measurements with units and build details. Confirm the GitHub link works and faculty can access the repository. Read the code and be ready to explain or modify it during verification.

Reference: ASSIGNMENT 1 [CO1].pdf, Tasks 1 to 12 and submission requirements, pages 1 to 13. No external fare rates or pass policies are asserted; policy assumptions are declared at the start of this report.

Independent Code Verification Record

Evidence type: actually executed assistant verification on 19 October 2026. This record is separate from the student execution fields and does not satisfy the required individual execution, screenshots or progressive GitHub commits. Predictions were prepared in the report before these checks. The outputs below are observations from these specific runs, not universal timing or type-size guarantees.

Environment: Windows; MinGW-W64 g++ 8.1.0, x86_64-posix-seh-rev0. Flags: -std=c++17 -Wall -Wextra -Wconversion -O2. The Task 9 driver was separately compiled with MinGW.org g++ 6.3.0 using the same flags. Both versions were identified from g++ --version. The compiler executable was run with its matching binary directory on PATH where needed.

Executed source	Observed result
types.cpp	sizeof bool/char/int/long long/unsigned/float/double = 1/1/4/8/4/4/8; double precision bits 53. int and long long limits matched the ranges stated in Task 3.
predict4.cpp	All four lines matched the ASCII prediction. Actual warning: comparison of integer expressions of different signedness: int and unsigned int [-Wsign-compare]. This warning is intentional in the supplied demonstration.
predict5.cpp	All five lines matched the prediction: 11 16; 9 6 4; 6 4; 3 1 -3 -1; 18.
todo6.cpp	All three lines matched the supplied expected output. No warning emitted.
procedural7.cpp and oop7.cpp	Both printed 1 9000, 1 5400, 0 5400 on separate lines. No warning emitted.
input8.cpp	Input lines abc, 25, 10 recovered with Invalid number, Invalid hour, then Accepted 10 Balance Rs 54.00.

Executed source	Observed result
fare9.cpp	T1 through T20 PASS; Five journey total Rs 96.00 balance Rs 104.00; Failures 0; exit status 0.
redesign12.cpp	R1 through R14 PASS; Failures 0; exit status 0. No warning emitted with the stated flags.

Observed accumulation results

Journeys	A double rupees	A error Rs	B double rupees	B error Rs
20,000	720000	0	1999.9999999992765	-7.2350303526036441e-10
500,000	18000000	0	49999.9999995529	-4.4710031943395734e-7
3,000,000	108000000	0	300000.00000195683	1.9568251445889473e-6

Observed integer totals matched every exact total in Task 10. int_fits printed 1, 1, 0 for workload A. This verifies the range analysis without triggering undefined overflow. The double error changed sign between 500,000 and 3,000,000 repetitions in workload B, illustrating that drift is not necessarily monotonic. Two-decimal output would have hidden all these measured errors.

Single-run combined A-loop times were approximately 0.0000166 s, 0.0003973 s and 0.002486 s. These are actual observations from one run, not repeated benchmark measurements; CPU model and scheduling were not recorded. They cannot support a fair integer-versus-double speed claim. Student benchmark fields remain blank for the required own measurements.

Interpretation: the executed functional checks support the stated calculation, boundary and rejection rules for the listed inputs. The integer ledger produced exact expected totals; the decimal stress test showed small binary rounding drift. Task 11 and Task 12 recommendations are therefore supported by these independent checks, while the student must still execute and document their own evidence.